

Case Study #2 Report

Student names: *Augustus Kaufmann, Aaron Duda*

Course: *Machine Learning* – Professor: *Dr. Sanmukh Kuppannagari*

Due date: *May 6th, 2026*

Chosen Clustering Function and Parameters

To select the best model suitable for determining the proper clustering of maritime vessel signals, we first considered using some of the algorithms in the scikit-learn module on clustering found at <https://scikit-learn.org/stable/modules/clustering.html>.

We began by comparing against the provided K-means baseline. The baseline uses all features except `OBJECT_ID` and `VID`, standardizes them, and runs K-means with $K = 20$. This is a reasonable starting point because `set1.csv` has 20 vessels, so the baseline is given the correct number of clusters for that dataset. However, even with the correct K , the baseline still performed poorly. This showed that knowing the number of clusters is not the same as correctly identifying vessel tracks. K-means minimizes distance to cluster centroids, so it tends to prefer compact, roughly circular clusters. AIS vessel observations are not necessarily shaped this way; one vessel can produce a long path over time, and nearby vessels can pass through similar areas.

We also tested the idea of using K-means together with silhouette score to estimate K . To estimate K , we first found the K value for which there was an elbow using the `kneed` library. After that, we pick the K value within ± 2 of the elbow which has the best average silhouette value. This method does successfully find the correct value for K , but this ultimately did not lead to a significant boost in performance, as we were still using the poorly performing K-means model from `scikit-learn`.

Other algorithms from the scikit-learn clustering page were considered conceptually. We did not choose Affinity Propagation or Mean Shift because they are less scalable with respect to the number of samples. We did not choose plain DBSCAN because it depends heavily on selecting one global `eps` value, which is difficult when vessel-report density varies. OPTICS, BIRCH, and Gaussian mixture models may also be reasonable comparisons, but we did not include them in our final implementation. We focused our validation on the provided K-means baseline and HDBSCAN parameter searches because HDBSCAN directly addresses the unknown-number-of-clusters requirement while still fitting the density-based structure of vessel tracks.

For these reasons, we chose HDBSCAN as our final clustering algorithm. Unlike K-means, HDBSCAN does not require us to specify `n_clusters` in advance. Instead, it estimates clusters from the density structure of the data, which is more suitable for the unknown number of vessels in `set3noVID.csv`.

To find the correct number of clusters and the correct cluster assignments, we set the following HDBSCAN parameters, varied as specified.

1. `min_cluster_size` (Varied)

This parameter represents the smallest group of observations that HDBSCAN is allowed to treat as a cluster. Since vessels can have very different numbers of AIS reports, this parameter is important for balancing small vessels against larger vessel tracks. We varied this parameter during validation and selected `min_cluster_size = 150` for the final model.

2. `min_samples` (Varied)

This parameter controls how conservative HDBSCAN is when deciding whether a point belongs to a dense region. Larger values make the algorithm stricter and can label more points as noise, while smaller values allow less dense vessel tracks to still form clusters. We varied this parameter and selected `min_samples = 3` for the final model.

3. `cluster_selection_method`

This parameter determines how clusters are selected from the density hierarchy. We considered the available options and used `eom`, which selects stable clusters from the hierarchy and gave strong validation performance.

4. Feature set and feature weights (Varied)

In addition to the HDBSCAN parameters, we varied which features were used and how strongly they were weighted after standardization. The best balanced final configuration used `SEQUENCE_DTTM`, `LAT`, and `LON`, with weights `[0.5, 3.0, 3.0]` for time, latitude, and longitude respectively.

Method for Unknown Number of Clusters

Our final method uses HDBSCAN to solve the unknown-number-of-clusters problem directly. Instead of choosing K , HDBSCAN builds a hierarchy of density-connected regions and selects stable clusters from that hierarchy. For our problem, each stable cluster is interpreted as a predicted vessel.

To tune our hyperparameters, we used a method similar to grid search, where we tried several combinations of different values of hyperparameters as well as different configurations of weighted features. We selected the hyperparameter and feature configuration which maximized the adjusted random index, (a metric which captures how well the model's cluster assignments are similar to the true assignments), for both dataset 1 and 2.

The following code shows the grid of feature configurations and HDBSCAN parameters that were tested during validation:

```
feature_configs = [
    {'features': ('time', 'position'), 'weights': {'time': 0.25, 'position': 3.0}},
    # etc. ...
]
min_cluster_sizes = [10, 25, 50, 75, 100, 150, 200]
min_samples_values = [3, 5, 10]
selection_methods = ['eom', 'leaf']
```

Feature Selection & Preprocessing

In this section, we briefly go through how we preprocessed the data, considering details such as dropped columns, extracted features, feature weighting, and our standardization method.

VID. To run clustering against the test data sets `set1` and `set2`, we drop the VID column so that it does not affect the clustering. After clustering we use the ground truth cluster assignments dictated by VID to determine the accuracy of the model. For `set3noVID.csv`, the VID values are unavailable for evaluation, so VID cannot be used as an input feature.

OBJECT_ID. We also drop the OBJECT_ID column, since it provides no relevant information to help with clustering vessel reports. It is only a unique identifier for each observation and does not describe vessel behavior.

SEQUENCE_DTTM. For this feature, we convert it into seconds. This is a form understandable to our model as opposed to the UTC `hh:mm:ss` format. We keep this feature because it represents when each vessel report occurred, which is important since vessels form paths over time.

LAT and LON. We keep LAT and LON because they represent where each vessel report occurred. These are the most important spatial features in the final model, since a vessel should form a continuous path through space over time.

SPEED_OVER_GROUND and COURSE_OVER_GROUND. We also tried including SPEED_OVER_GROUND and COURSE_OVER_GROUND. For course, we converted the angle into sine and cosine components so that directions near 0° and 360° would be treated as close. However, validation showed that adding speed and course reduced performance compared with using time and position only. This may be because speed and course can be noisy or can vary along a single vessel's route, causing one vessel to be split into multiple clusters.

Standardization. To ensure feature scale did not affect the outcome of the clustering, we use the `StandardScaler` from `scikit-learn` to transform the data such that each feature has its mean set to 0 and variance to 1.

Feature Weighting. After standardization, our final predictor applies feature weights to SEQUENCE_DTTM, LAT, and LON:

[0.5, 3.0, 3.0]

for time, latitude, and longitude respectively. This gives spatial position more influence than time while still preserving temporal ordering. We chose this weighting because validation showed that position was more important for separating vessel track. Even though time was still useful, it should not dominate calculations.

Validation Results

We evaluated performance using adjusted Rand index on the labeled datasets `set1.csv` and `set2.csv`.

Method	set1.csv ARI	set2.csv ARI
K-means baseline	0.2422	0.3335
Initial HDBSCAN, all features equal weight	0.3755	0.8268
Final tuned HDBSCAN	0.6975	0.9817

The final tuned HDBSCAN model substantially improves over the baseline on both labeled datasets. The largest improvement is on `set2.csv`, but `set1.csv` also improves by a large margin. This suggests that the final feature selection and parameter tuning better match the structure of the AIS vessel-tracking problem than the original K-means baseline.

Insights and Failed Attempts

Perhaps the most surprising insight from this study was the fact that more data does not necessarily make your model better. In the case of the maritime vessels data, it was more beneficial to remove columns concerned with the vessel's speed and only retain information about time and position. This fact is likely due to the noisy nature of vessel speed, making the separability of the clusters more difficult when the data is included.

We also found it interesting that K-means was mostly ineffective at solving this clustering problem. In our first attempt, we thought that if we could obtain the correct value for K using the elbow method and silhouette values, we could simply tune the other hyperparameters to build a successful model. Below is the code for determining the K value using the elbow and the silhouette values.

```
# elbow method on inertia
knee = KneeLocator(list(map(int, ks)), inertias, curve='convex',
                    direction='decreasing')
elbow_k = knee.knee if knee.knee is not None else ks[np.argmax(
    sil_scores)]

# pick K closest to elbow that also maximizes silhouette within a
# window around the elbow
window = 2
elbow_idx = np.where(ks == elbow_k)[0][0]
lo = max(0, elbow_idx - window)
hi = min(len(ks), elbow_idx + window + 1)
best_index = lo + np.argmax(sil_scores[lo:hi])
```

But nothing we did could improve the model above the baseline. Pursuing this solution was an oversight on our part since the baseline shows that even if you know the exact value for K, K-means still cannot cluster the data correctly.